

ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."

— Spandan Prayas Patra

TABLE OF CONTENTS

- Chapter 1: Introduction & Philosophical Architecture
 - Chapter 2: Variables, Optional Types & Expression Syntax
 - Chapter 3: The 3-Level Native Interactivity Architecture
 - Chapter 4: Control Flow & Iteration
 - Chapter 5: Functions, Natural Declarations, Interfaces & Async / Await
 - Chapter 6: Pattern Matching & Decision Engine
 - Chapter 7: Exception Handling & Error Control
 - Chapter 8: Built-in Collections — Lists, Arrays, Dictionaries, Sets
 - Chapter 9: String Manipulations, Math & DateTime Primitives
 - Chapter 10: File I/O, System Operations & Security
 - Chapter 11: Frontend Structural Markup (`.enlgf` → HTML5)
 - Chapter 12: Styling & Design Systems (`.enlgd` → CSS3)
 - Chapter 13: Client-Side Scripting (`.enlgs` → JavaScript ES6+)
 - Chapter 14: Database Schemas & Queries (`.enlgdb` → SQL)
 - Chapter 15: Native NLP & AI Primitives
 - Chapter 16: EnLang Web Server & Multi-File Architecture
 - Chapter 17: Canonical Grammar Rules, Order & Syntax Layout Specification
 - Chapter 18: Complete Production Case Study — Lumina Workspace
 - Appendix A: Universal Natural Syntax Cheatsheet
 - Appendix B: CLI & EnLang Package Manager (EPM) Manual
-

CHAPTER 1: INTRODUCTION & PHILOSOPHICAL ARCHITECTURE

1.1 The Vision of Natural Programming

For over seven decades, software development has required human engineers to adapt their thoughts to artificial syntaxes, cryptic symbols (`{}`, `;`, `=>`, `::`), and unforgiving compiler rules.

EnLang was designed by **Spandan Prayas Patra** to invert this paradigm: to allow developers to write application logic in **pure, clear, expressive natural English**, while maintaining **100% deterministic compilation** into battle-tested native target languages (Python 3, HTML5, CSS3, JavaScript ES6+, SQL).

EnLang is **not** a pseudo-code generator or an LLM wrapper. It is a **deterministic, multi-target compiler and runtime engine**.

CHAPTER 2: VARIABLES, OPTIONAL TYPES & EXPRESSIONS

2.1 Natural Expressive Flexibility

EnLang supports multiple natural syntaxes for variable assignment:

```
set score to 100
let score = 100
store 100 in score
score is 100
```

CHAPTER 5: FUNCTIONS, NATURAL DECLARATIONS, INTERFACES & ASYNC / AWAIT

5.1 Flexible Function Declarations & Invocations

EnLang supports both standard parameter signatures and expressive natural English declarations:

Standard Declaration

```
function print_numbers(n):
  if n is greater than 10 then:
    return
  display n
  print_numbers(n plus 1)

print_numbers(1)
```

Expressive Natural English Declaration & Call (v2.0)

```
function numbers using n:
  if n is greater than 10 then:
    return
  display n
  call numbers with (n plus 1)

start numbers from 1
```

Supported Natural Syntaxes:

- Declarations: `function foo using n:`, `function foo taking n:`, `action foo given n:`, `task foo for n:`
- Invocations: `start foo from 1`, `start foo with 1`, `call foo with 1`, `run foo using 1`

CHAPTER 17: CANONICAL GRAMMAR RULES, ORDER & SYNTAX LAYOUT SPECIFICATION (v2.0)

17.1 Function Declarations & Invocations

Syntax Style	Function Header	Execution Call	Transpiled Output
Standard	<code>function foo(n):</code>	<code>foo(1)</code>	<code>def foo(n): / foo(1)</code>
Natural English	<code>function foo using n:</code>	<code>start foo from 1</code>	<code>def foo(n): / foo(1)</code>
Directive	<code>action foo given n:</code>	<code>call foo with 1</code>	<code>def foo(n): / foo(1)</code>
Task	<code>task foo for n:</code>	<code>run foo using 1</code>	<code>def foo(n): / foo(1)</code>

APPENDIX A: UNIVERSAL SYNTAX REFERENCE

Category	EnLang Syntax Example	Native Transpilation
Natural Function	function numbers using n:	def numbers(n):
Natural Call	start numbers from 1/call numbers with 1	numbers(1)

Copyright © 2026 Spandan Prayas Patra. All rights reserved.
Published under the Open EnLang Specification License.